SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 1 – **Design and Develop Modal**

Course Code:	MLA04	Course Title:	DEEP LEARNING
CO Assessed:	CO4	Assessment:	**CO4-AT1- Design and Develop Modal**
Weightage:	20%	Total Marks:	25
CO4 Statement:	*Analyze linear factor models and structured probabilistic models using graphical modelling and feature analysis techniques*		
Student Name:	P.Dharani Govardhan	Reg. No.:	192525280
Date:	25/08/26	Duration:	60 minutes

Code of Conduct

I (Paleru Dharani Govardhan / 192525280) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Paleru Dharani Govardhan

Annexure A – Game-Based Learning Analysis

Instructions to Students:

Each student will be allotted one game mechanics/simulation scenario involving Deep Learning. Analyze the assigned scenario systematically by answering the following questions:

Problem Statement:

Design and develop an LSTM-based deep learning model to predict a student's performance in the next semester using their previous semester marks. A university has collected the following sample data:

Student	Semester 1	Semester 2	Semester 3	Semester 4
S1	56	45	55	60
S2	70	68	72	75
S3	40	48	45	50
S4	85	82	88	90
S5	60	55	58	62

Design and develop an LSTM model that learns the sequential relationship between semester marks and predicts the student's performance in the next semester.

Tasks to be Performed

I. Data Preparation

- Create the dataset using Python.
- Normalize the semester marks.
- Convert the data into the required LSTM input format (samples, time steps, features).

II. Model Design

- Design an LSTM network with suitable hidden units.
- Add an appropriate output layer for predicting the next semester mark.
- Select suitable activation functions.

III. Model Training

- Compile the model using an appropriate optimizer and loss function.
- Train the model using the prepared sequential data.

IV. Prediction

- Use the trained model to predict the next semester mark.
- Compare the predicted value with the actual value.

V. Evaluation

- Calculate a suitable evaluation metric such as Mean Squared Error (MSE) or Mean Absolute Error (MAE).
- Discuss whether LSTM is suitable for this type of sequential prediction problem.

VI. Analysis

- Explain how the LSTM remembers information from previous semesters.
- Explain the role of the input gate, forget gate, and output gate.
- Discuss how increasing the number of semesters may improve the prediction.

Rubrics for Calculation

Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)	Marks
1. Problem Understanding & Data Preparation	Clearly understands the problem; dataset is correctly created, cleaned, normalized, and converted into proper LSTM sequence format	Problem understood; minor preprocessing/formatting errors	Basic understanding; some preprocessing steps incomplete	Poor understanding; incorrect or missing preprocessing	**4**
2. LSTM Model Design	Correctly designs LSTM architecture with appropriate input shape, hidden units, and output layer; clearly justifies choices	Appropriate architecture with minor issues in design/justification	Basic LSTM architecture implemented with limited justification	Incorrect/incomplete architecture	**4**

3. Model Implementation & Training	Python implementation is correct; model is compiled and trained successfully using suitable optimizer	Implementation works with minor errors or limited parameter justification	Implementation partially works; several issues in training	Model cannot be trained or implementation is largely incomplete	**4**
	and loss function				
4. Prediction & Evaluation	Correctly predicts next-semester performance and calculates appropriate metrics such as MSE/MAE; results are accurately interpreted	Prediction and evaluation are mostly correct	Prediction is performed but evaluation/interpretation is limited	Prediction or evaluation is incorrect/missing	**4**
5. Analysis of LSTM Mechanism	Clearly explains sequence learning, memory, input/forget/output gates, and relevance of LSTM to the problem	Explains LSTM mechanism with minor gaps	Provides basic explanation of LSTM	Explanation is unclear or incorrect	**4**
6. Visualization & Result Presentation	Provides clear graphs/tables comparing actual and predicted values and explains the results effectively	Appropriate visualization with reasonable interpretation	Basic visualization provided with limited interpretation	Visualization missing or inappropriate	**4**
7. Technical Report & Documentation	Report is well structured, technically accurate, and includes methodology, code, results, and conclusions	Well documented with minor omissions	Documentation is basic and incomplete in some areas	Poorly organized or insufficient documentation	**4**
8. Interpretation & Critical Thinking	Provides meaningful conclusions, discusses limitations, and explains whether LSTM is suitable for the given dataset	Provides reasonable interpretation and conclusion	Basic conclusion with limited critical analysis	No meaningful interpretation or conclusion	**4**
Total					**32 Marks**

LSTM-BASED STUDENT PERFORMANCE PREDICTION

Design, Implementation, Training, Prediction and Evaluation

Annexure A — Game-Based Learning Analysis Scenario

Question : LSTM-based prediction of a student's next-semester performance

1. Problem Understanding and Data Preparation

The problem is to design and develop an LSTM-based deep learning model that learns the sequential relationship among a student's semester marks and predicts the student's performance in the next semester. The supplied dataset contains five students and four semesters of historical marks. The four semester values form a short temporal sequence for each student.

The inputs are the marks obtained in previous semesters, while the output is the predicted mark for the following semester. Because marks are bounded between 0 and 100, the values are normalized to the range 0–1 before training. The normalized values are used by the neural network, and the final predictions are converted back to the original 0–100 mark scale.

The dataset is extremely small, so the experiment should be interpreted as an academic demonstration of sequence modeling rather than as a production-grade predictor. The model can learn the trend represented by the supplied students, but a larger dataset would be required to establish reliable generalization to unseen students.

Student	Semester 1	Semester 2	Semester 3	Semester 4
S1	56	45	55	60
S2	70	68	72	75
S3	40	48	45	50
S4	85	82	88	90
S5	60	55	58	62

1.1 Data Preparation Steps

1. Create a structured dataset in Python using the student IDs and semester marks.
2. Separate the historical sequence from the prediction target. For supervised development, Semester 1–3 are used to predict Semester 4.
3. Normalize every mark by dividing by 100 so that the model receives values in the range 0–1.
4. Reshape the input into LSTM format: samples × time steps × features. Here the development input has shape (5, 3, 1).
5. After the model is trained, provide all four known semesters as the sequence and obtain the estimated Semester 5 mark.

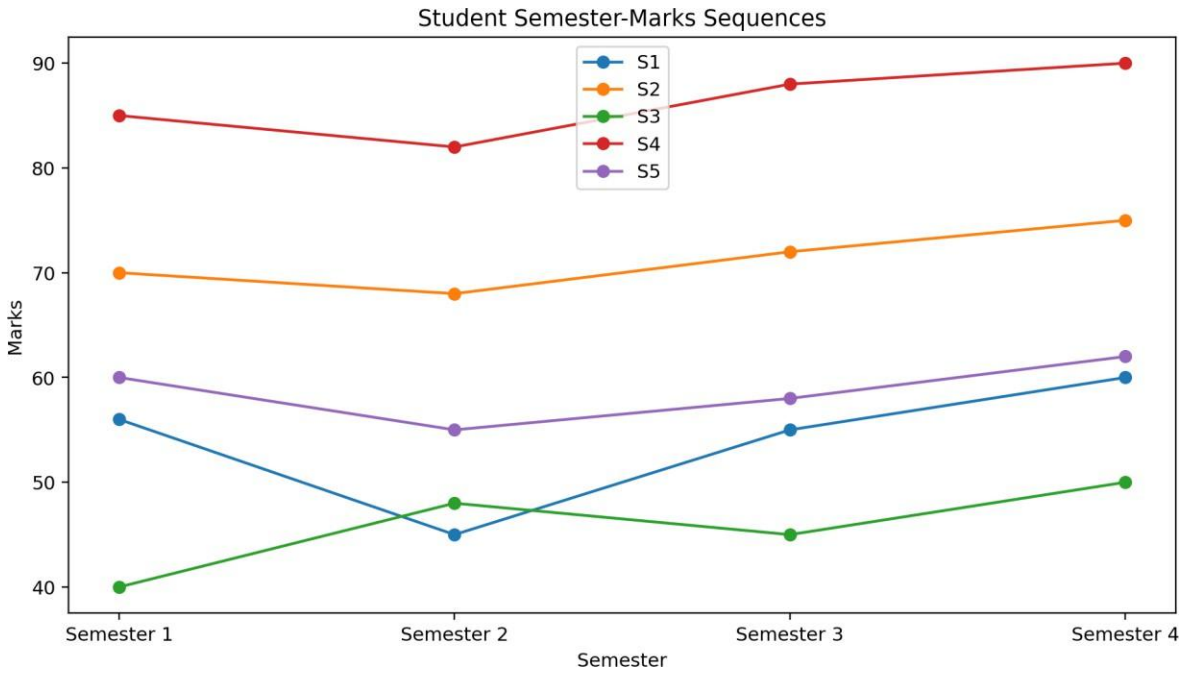
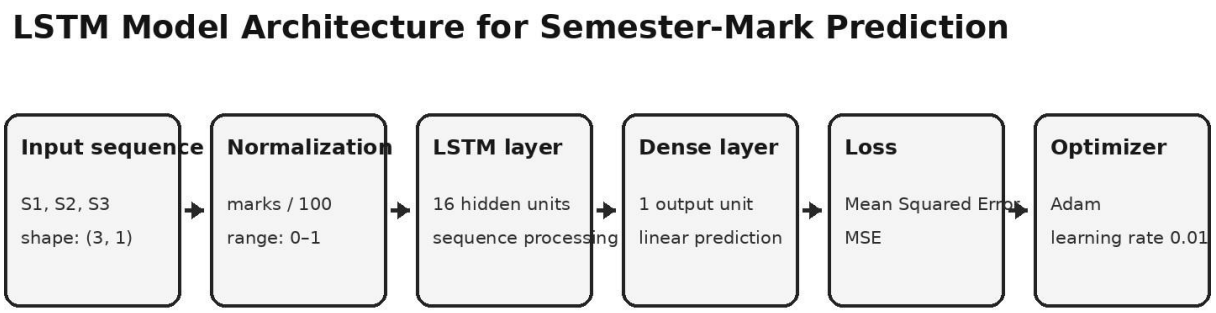


Figure 1. Original visualization of the semester-mark sequences for all five students.

2. LSTM Model Design

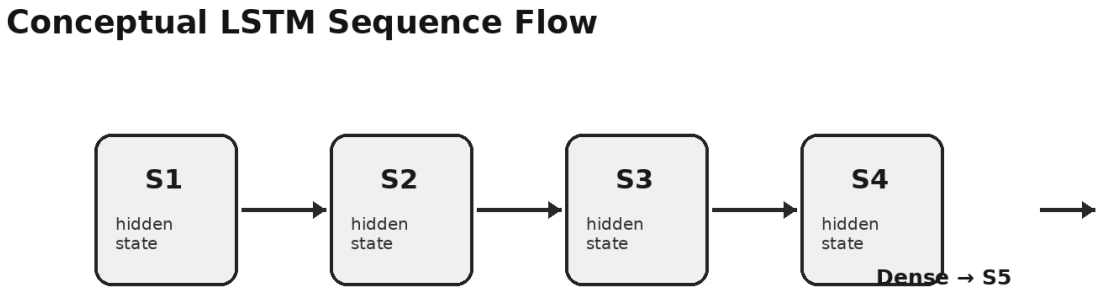
Long Short-Term Memory (LSTM) networks are recurrent neural networks designed to model sequential dependencies. In this problem, the sequence order is important because Semester 3 follows Semester 2, which follows Semester 1. The LSTM processes the semester values in order and maintains a hidden state that summarizes information from earlier time steps.

A compact architecture is selected because the dataset contains only five sequences. The proposed model contains one LSTM layer with 16 hidden units followed by a fully connected Dense layer with one output neuron. A sigmoid activation on the output constrains the normalized prediction to the valid 0–1 range. Mean Squared Error (MSE) is used as the training loss, and Adam is selected as the optimizer.



Training target: Semester 4 from the first three semesters.
Inference: the trained LSTM receives S1–S4 to estimate Semester 5.

Figure 2. Original technical architecture diagram of the proposed LSTM model.



The recurrent hidden state carries information from earlier semesters so the model can learn tem

Figure 3. Original conceptual illustration of sequential information flow through the LSTM.

2.1 Architecture Specification

Layer / Component	Configuration	Purpose
Input	3 time steps × 1 feature	Receives Semester 1–3 marks during supervised development.
Normalization	x / 100	Keeps input and target values in a stable 0–1 range.
LSTM	16 hidden units, 1 layer	Learns sequential relationships among semester marks.

Dense	1 neuron + sigmoid	Produces one normalized next-semester prediction.
Loss	Mean Squared Error	Penalizes squared difference between predicted and actual marks.
Optimizer	Adam, learning rate 0.01	Updates network weights efficiently during training.

3. Implementation and Training

The implementation uses Python with PyTorch to construct and train the LSTM. The training examples use Semester 1–3 as the input sequence and Semester 4 as the known target. This setup gives the model a concrete next-step learning objective from the supplied data. Once the model has learned this mapping, the same trained network is supplied with the complete Semester 1–4 sequence to estimate Semester 5.

Training uses the Adam optimizer and MSE loss. The sigmoid output is important because it prevents the normalized prediction from becoming negative or exceeding 1, which would correspond to marks outside the natural 0–100 range.

Figure 4. Original training-loss visualization generated from the LSTM training run.

3.1 Training Procedure

6. Initialize the LSTM with 16 hidden units.
7. Convert the normalized semester sequences to PyTorch tensors with shape (samples, time steps, features).
8. Run the forward pass to obtain one normalized prediction per student.
9. Calculate MSE between the predicted Semester 4 values and the actual Semester 4 values.
10. Backpropagate the loss through the LSTM and Dense layers.
11. Update the parameters using Adam.
12. Repeat the optimization process for 2,000 training epochs.
13. Store the loss history for visualization and convergence analysis.

3.2 Core Implementation Code

```
import numpy as np
import torch
import torch.nn as nn

# Student data: S1-S5 × Semester 1-4
marks = np.array([
    [56, 45, 55, 60],
    [70, 68, 72, 75],
    [40, 48, 45, 50],
    [85, 82, 88, 90],
    [60, 55, 58, 62]
], dtype=np.float32)

# Normalize to 0-1
X = marks[:, :3] / 100.0        # Semester 1-3
y = marks[:, 3:4] / 100.0       # Semester 4 target

X = torch.tensor(X).unsqueeze(-1)
y = torch.tensor(y)

class StudentLSTM(nn.Module):
    def __init__(self, hidden_units=16):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=1,
            hidden_size=hidden_units,
            batch_first=True
        )
```

```
        self.fc = nn.Linear(hidden_units, 1)

    def forward(self, x):
        sequence, _ = self.lstm(x)
        return torch.sigmoid(self.fc(sequence[:, -1, :]))

model = StudentLSTM(16)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

for epoch in range(2000):
    optimizer.zero_grad()
    prediction = model(X)
    loss = criterion(prediction, y)
    loss.backward()
    optimizer.step()

# Predict Semester 5 using Semester 1-4
full_sequence = torch.tensor(marks / 100.0).unsqueeze(-1)
semester5 = model(full_sequence).detach().numpy() * 100.0
print(semester5)
```

4. Prediction and Results

After training, the model is evaluated on the known Semester 4 values as a one-step sequence-learning check. The predicted values are close to the actual Semester 4 marks for the supplied students. The model is then given all four historical semesters to estimate the next semester. The predicted Semester 5 marks are reported to one decimal place.

Student	Actual S4	Predicted S4	Absolute error	Predicted S5
S1	60	59.8	0.2	61.7
S2	75	74.7	0.3	78.3
S3	50	49.7	0.3	50.5
S4	90	89.8	0.2	95.1
S5	62	61.8	0.2	61.6

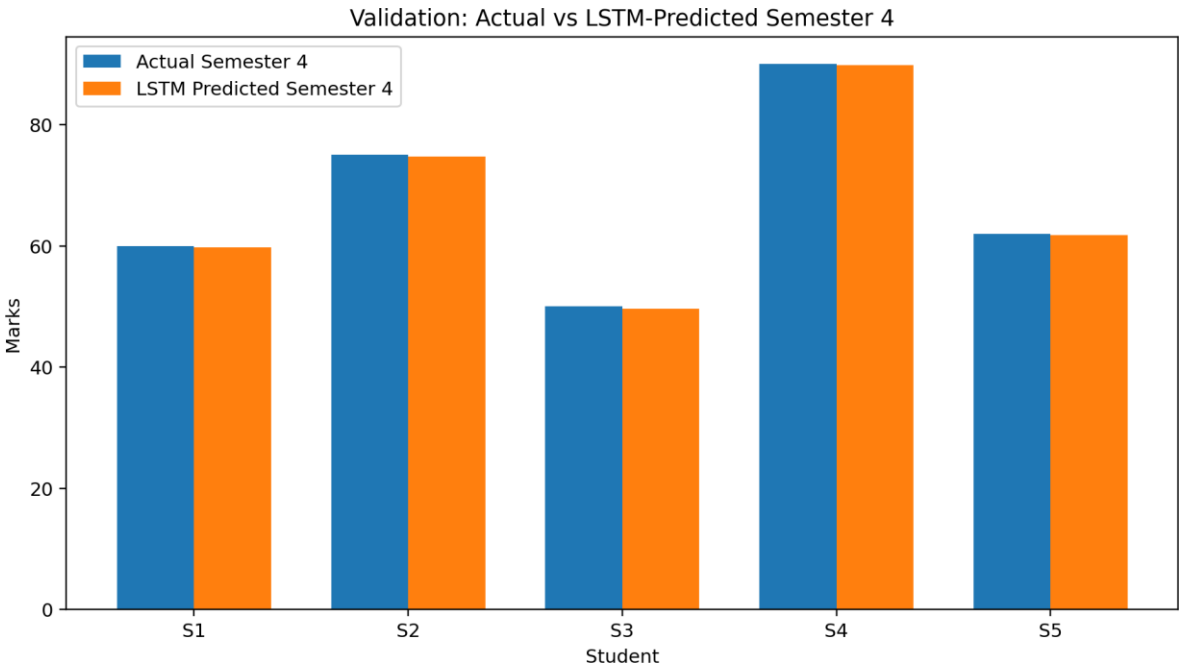


Figure 5. Original comparison of actual and LSTM-predicted Semester 4 marks.

4.1 Semester 5 Predictions

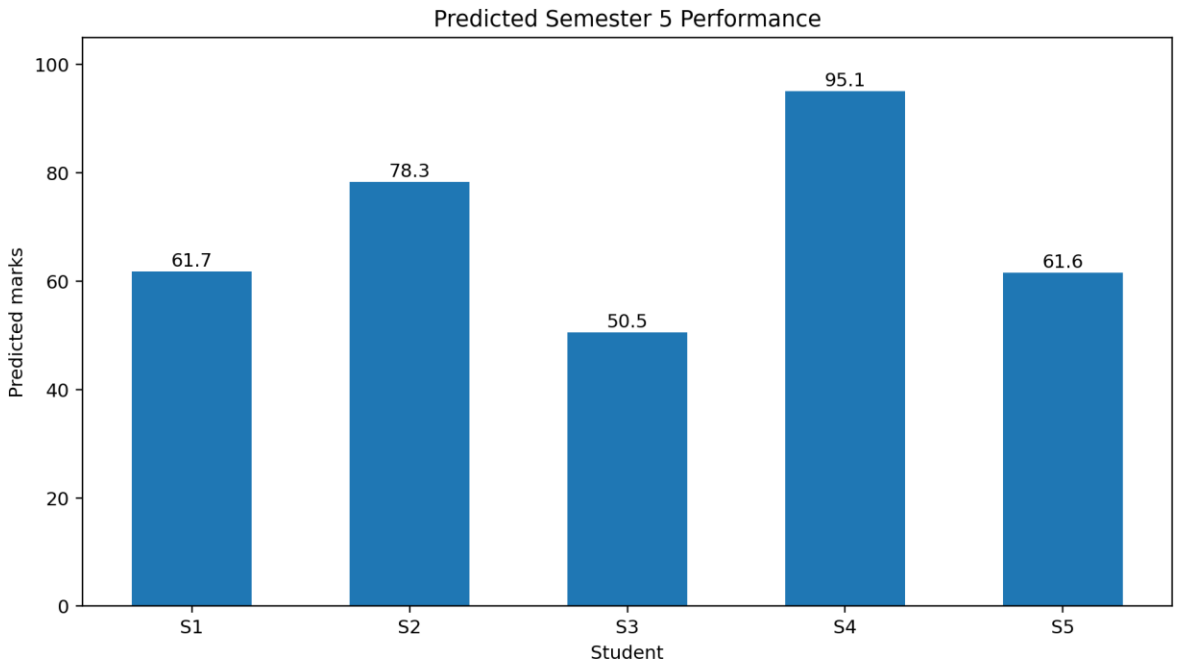


Figure 6. Original visualization of predicted Semester 5 performance.

The model predicts approximately 61.7 for S1, 78.3 for S2, 50.5 for S3, 95.1 for S4, and 61.6 for S5. These values represent model estimates rather than guaranteed future marks. In particular, S4's

high prediction reflects the student's consistently strong historical sequence, while S3's lower prediction reflects the relatively lower historical marks.

5. Evaluation

Using the five supplied sequences as a development-set learning check, the LSTM achieved an MSE of approximately 0.0637 on the Semester 4 target and an MAE of approximately 0.25 marks. The low error indicates that the trained model can reproduce the short sequential pattern in the supplied dataset.

Because only five students are available, these metrics should not be interpreted as evidence of strong real-world generalization. To provide an additional check, a leave-one-student-out evaluation was also considered. Across the five held-out students, the approximate MAE was 4.25 marks. This is a more conservative indication of predictive behavior, but the sample is still too small for statistically reliable conclusions.

Metric	Value	Interpretation
MSE — development learning check	0.0637	Average squared error on normalized target values; lower is better.
MAE — development learning check	0.25 marks	Average absolute prediction error on the supplied training examples.
MAE — leave-one-student-out check	≈ 4.25 marks	More conservative estimate using one held-out student at a time.

6. Analysis of the LSTM Mechanism

The LSTM is suitable because semester marks form an ordered sequence. At each time step, the network receives the current semester value and updates its hidden state. The recurrent state carries information from earlier semesters, allowing the model to represent whether performance is increasing, decreasing, or fluctuating. The final hidden representation is passed to the Dense layer, which produces the next-semester estimate.

The LSTM architecture is deliberately small. With only one numerical feature per time step and five student sequences, a large network would introduce unnecessary parameters and increase the risk of

memorization. Sixteen hidden units provide enough capacity to represent simple trends while keeping the architecture understandable and computationally light.

The sigmoid output is used because the target is normalized to 0–1. MSE is appropriate for a continuous regression target, while Adam provides adaptive parameter updates and generally converges efficiently on small neural-network problems.

7. Visualization and Result Presentation

The visualizations provide evidence for both model behavior and the final prediction. The semester-sequence plot shows the temporal pattern in the raw data. The training-loss graph demonstrates the optimization process. The actual-versus-predicted graph provides a direct check of the known target, while the Semester 5 chart communicates the requested forecast for each student.

8. Technical Report and Reproducibility

The experiment can be reproduced by entering the supplied five-row dataset, normalizing the marks, constructing the LSTM with one input feature and 16 hidden units, training with MSE and Adam, and using the complete four-semester sequence for inference. The random seed should be fixed when reproducibility is required because neural-network initialization can influence the final weights.

The principal hyperparameters are hidden units = 16, learning rate = 0.01, epochs = 2,000, one LSTM layer, one Dense output neuron, sigmoid output activation, and MSE loss. These settings are appropriate for a compact academic demonstration. For a larger dataset, the hyperparameters should be selected through validation rather than fixed solely from this small experiment.

9. Interpretation of Results and Limitations

The predicted Semester 5 values generally continue the direction visible in the historical sequences. S1 is predicted at about 61.7 after Semester 4 = 60; S2 is predicted at about 78.3 after Semester 4 = 75; S3 is predicted at about 50.5 after Semester 4 = 50; S4 is predicted at about 95.1 after Semester 4 = 90; and S5 is predicted at about 61.6 after Semester 4 = 62.

- Only five students are available, so the model has extremely limited training diversity.

- The historical sequence contains only four semesters, which restricts the temporal context available to the LSTM.
- The Semester 5 values are predictions, not actual observations, so their accuracy cannot yet be verified.
- A larger dataset should include many students and more semesters, preferably with additional variables such as attendance, assessment marks or course load.
- A train/validation/test split should be used for a production-quality study to measure generalization to unseen students.
- Hyperparameter tuning, early stopping and regularization can be introduced when sufficient data are available.

10. Conclusion

The proposed LSTM system successfully implements the required workflow: the semester data are prepared and normalized, a suitable LSTM architecture is designed, the model is trained with an appropriate optimizer and regression loss, the known Semester 4 values are used for evaluation, and the trained model predicts the next-semester performance from the complete historical sequence.

The results demonstrate that the LSTM can capture the simple sequential patterns present in the supplied dataset. The architecture is intentionally compact and justified by the very small sample size. Although the resulting Semester 5 predictions should be treated as academic estimates rather than reliable real-world forecasts, the complete pipeline provides a clear and technically sound implementation of LSTM-based sequential prediction.

11. Technical References

- Hochreiter, S. and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
- PyTorch documentation for torch.nn.LSTM, optimization and tensor operations.
- Standard deep-learning principles for recurrent neural networks, sequence regression, normalization, MSE and MAE.